

Cluster Image Portal — Architecture & Sequences

Web GUI component of `imgctl` v2.2.0 · NVIDIA BCM / DGX head node

The one idea: a **root producer** collects images on a timer and atomically writes a JSON **snapshot**; an **unprivileged web service** only *reads* that snapshot. They share one file and nothing else — so the public, no-auth web tier never runs `imgctl`, never SSHes, and never sees Harbor credentials.

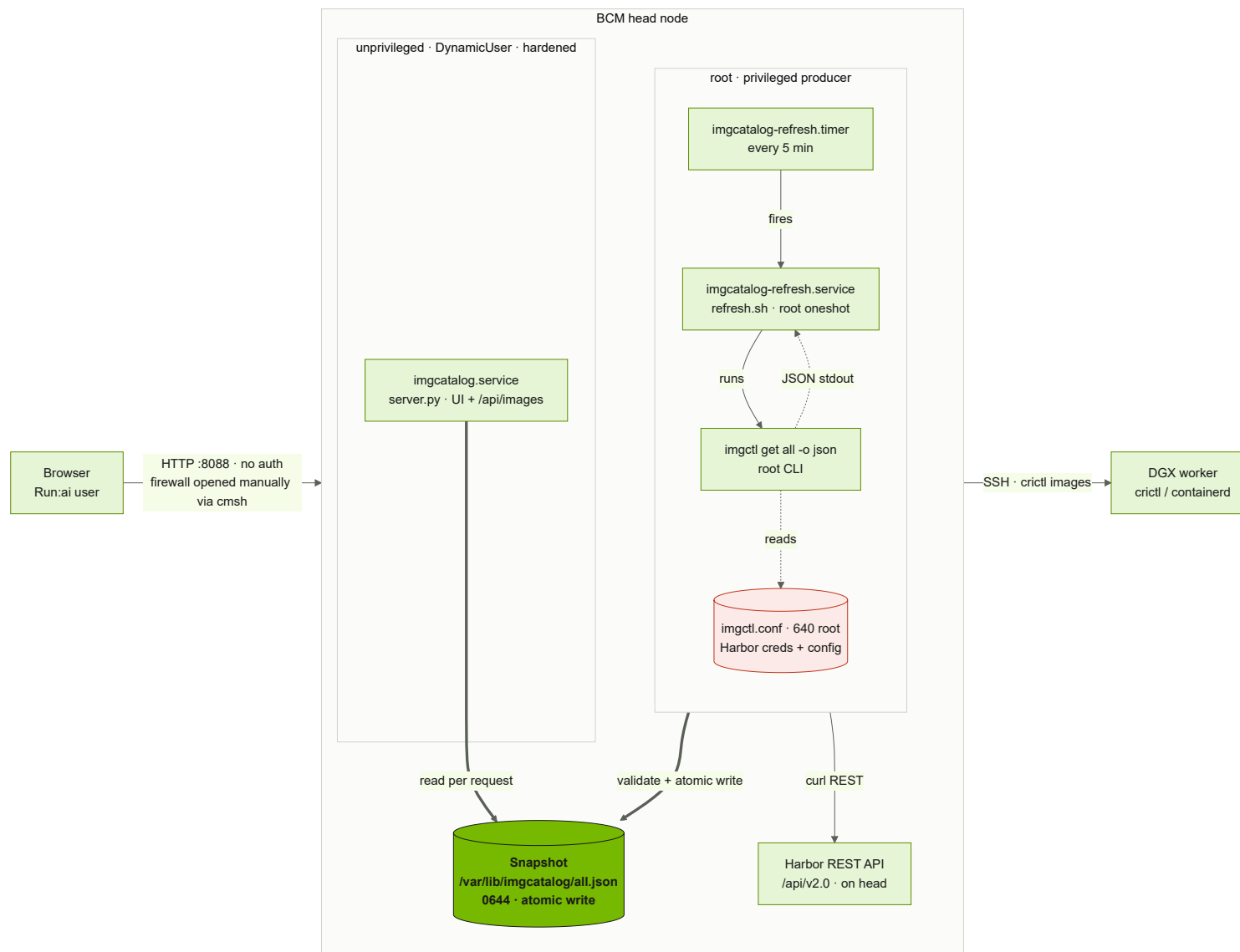
Components & responsibilities

Component	Runs as	Triggered by	Reads	Writes	Purpose
<code>imgcatalog-refresh.timer</code>	systemd	boot + every 5 min	—	—	Fires the refresh oneshot (<code>Persistent=true</code>)
<code>refresh.service</code> → <code>refresh.sh</code>	root (oneshot)	the timer	<code>imgctl.conf</code>	the snapshot	Run <code>imgctl</code> , validate, atomically publish
<code>imgctl</code>	root (CLI)	<code>refresh.sh</code>	<code>imgctl.conf</code>	stdout → temp	Query Harbor API + <code>crictl</code> over SSH
Harbor REST API	service on head	<code>imgctl</code>	—	—	Source: custom/registry images
DGX worker <code>crictl</code>	on worker (via SSH)	<code>imgctl</code>	—	—	Source: node-cached NGC/Docker images
Snapshot <code>all.json</code>	file (<code>0644</code>)	—	by web tier	by <code>refresh.sh</code>	The decoupling seam (one JSON file)
<code>imgcatalog.service</code> → <code>server.py</code>	unprivileged <code>DynamicUser</code>	always-on (<code>Restart=always</code>)	the snapshot	—	Serve UI + JSON API
Browser	client	user	<code>/</code> , <code>/api/images</code>	—	Browse + copy pull reference

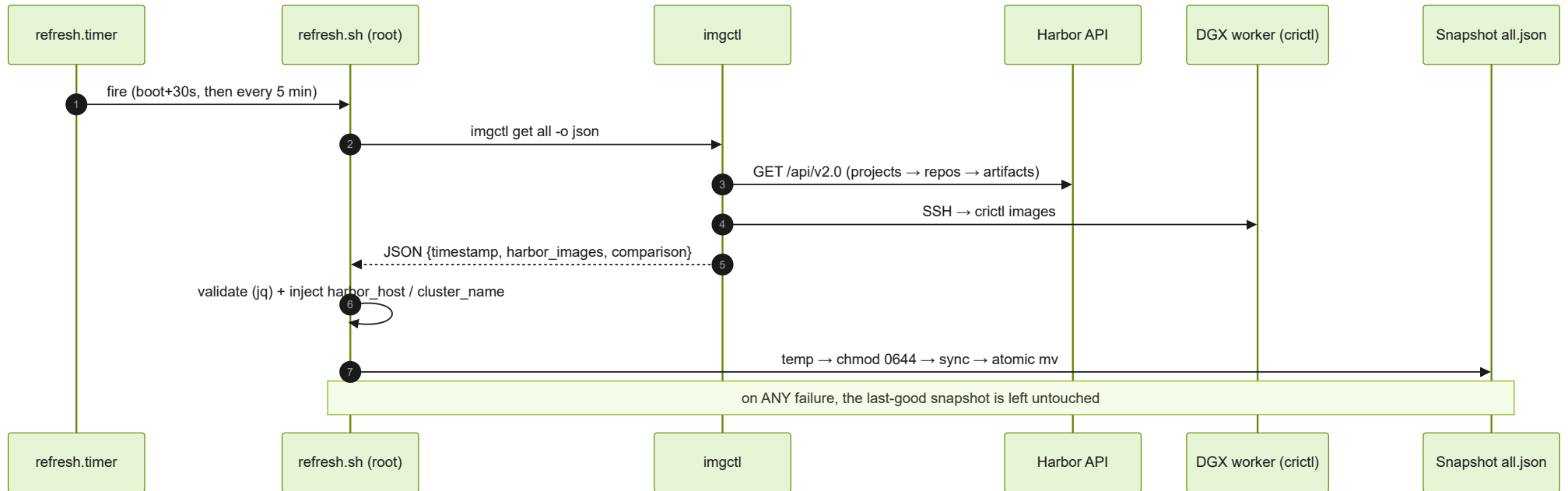
API: `GET /` (UI) · `GET /api/images` (flattened JSON, exact `reference` per image) · `GET /api/raw` · `GET /healthz` · `GET /version`. Non-GET/HEAD → `405`. | **Ops:** snapshot `/var/lib/imgcatalog/all.json`; install via `install.sh`; firewall opened **manually** via `cmsh ... openports` (never hand-edit `/etc/shorewall/rules`).

Security: no authentication; binds `0.0.0.0:8088` (plain HTTP) — access gated **only by the firewall port**. The web tier is sandboxed (`DynamicUser` , `NoNewPrivileges` , empty `CapabilityBoundingSet` , `ProtectSystem=strict` , `MemoryMax` / `CPUQuota` / `TasksMax`) and read-only — it exposes image names/tags/sizes only.

1 · Architecture



2 · Sequence — refresh path (root producer, periodic)



3 · Sequence — request path (unprivileged web tier)

